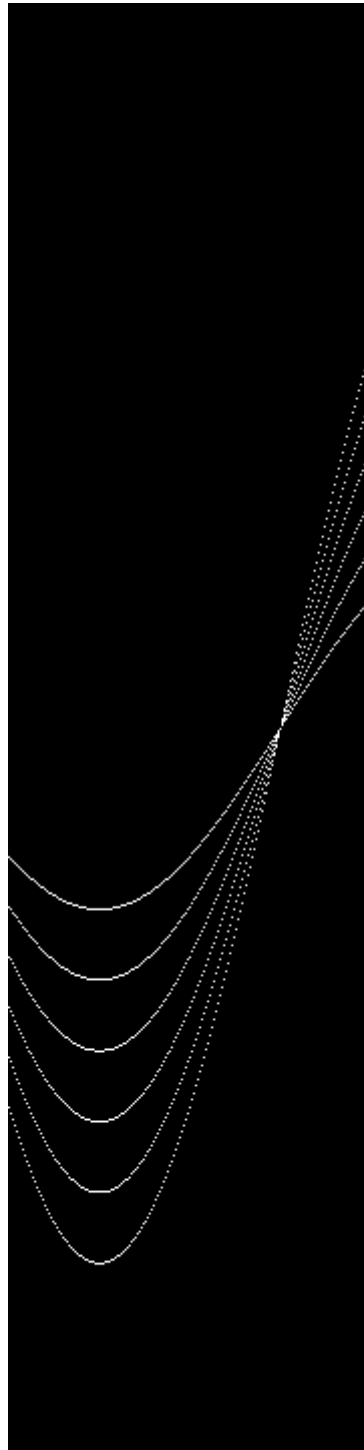


課題 1



```
import cv2
```

```
import numpy as np
```

```
import math
```

```
image = cv2.imread('image09-1.png', cv2.IMREAD_GRAYSCALE)
```

```
h, w = image.shape
```

```
rho_max = int(math.sqrt(h * h + w * w))
```

```
image2 = np.zeros((rho_max * 2, 180), np.uint8)
```

```
count = 0
```

```
for y in range(h):
```

```
    for x in range(w):
```

```
        if image[y][x] == 255:
```

```
            count += 1
```

```
        if count % 20 == 0:
```

```
            for theta in range(180):
```

```
                rad = math.radians(theta)
```

```
                rho = int(x * math.cos(rad) +
```

```
                           y * math.sin(rad))
```

```
rho_index = rho + rho_max
```

```
image2[rho_index][theta] = 255
```

```
if count >= 100:
```

```
    break
```

```
if count >= 100:
```

```
    break
```

```
cv2.imshow("rho-theta space", image2)
```

```
cv2.imwrite("result9-2.png", image2)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

プログラムの解説

課題 1 では、入力画像の直線を構成する数点を選び、それぞれの点を $\rho - \theta$ 空間へ変換した。変換には $\rho = x \cos \theta + y \sin \theta$ を用いた。変換後の結果を見ると、1 つの点は $\rho - \theta$ 空間で曲線として表されることが確認できた。また、同じ直線上にある点の曲線は共通する位置を通ることが分かった。これにより、Hough 変換では $\rho - \theta$ 空間を利用して直線を検出できることを確認した。

考察

課題 1 では、直線上の点を $\rho - \theta$ 空間に変換した。その結果、1 つの点は曲線として表されることが分かった。また、同じ直線上の点から得られた曲線は共通する位置を通ることが確認できた。このことから、Hough 変換では $\rho - \theta$ 空間を利用して直線を検出していることが分かった。

課題 2

```
import cv2
```

```
import numpy as np
```

```
import math
```

```
image = cv2.imread('image09-1.png', cv2.IMREAD_GRAYSCALE)
```

```
h, w = image.shape
```

```
rho_max = int(math.sqrt(h * h + w * w))
```

```
vote = np.zeros((rho_max * 2, 180), np.uint32)
```

```
image2 = np.zeros((rho_max * 2, 180), np.uint8)
```

```
for y in range(h):
```

```
    for x in range(w):
```

```
        if image[y][x] == 255:
```

```
            for theta in range(180):
```

```
                rad = math.radians(theta)
```

```
                rho = int(x * math.cos(rad) +
```

```
                        y * math.sin(rad))
```

```
                rho_index = rho + rho_max
```

```
                vote[rho_index][theta] += 1
```

```
max_vote = np.max(vote)
```

```
for rho in range(rho_max * 2):
```

```
    for theta in range(180):
```

```
        image2[rho][theta] = int(
```

```
            vote[rho][theta] * 255 / max_vote
```

```
        )
```

```
best_rho = 0
```

```
best_theta = 0
```

```
max_count = 0
```

```
for rho in range(rho_max * 2):
```



```
for theta in range(180):
```

```
    if vote[rho][theta] > max_count:
```

```
        max_count = vote[rho][theta]
```

```
        best_rho = rho - rho_max
```

```
        best_theta = theta
```

```
print("rho =", best_rho)
```

```
print("theta =", best_theta)
```

```
cv2.imshow("rho-theta space", image2)
```

```
cv2.imwrite("result9-2.png", image2)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

出力

```
rho = 0
```

```
theta = 135
```

プログラムの解説

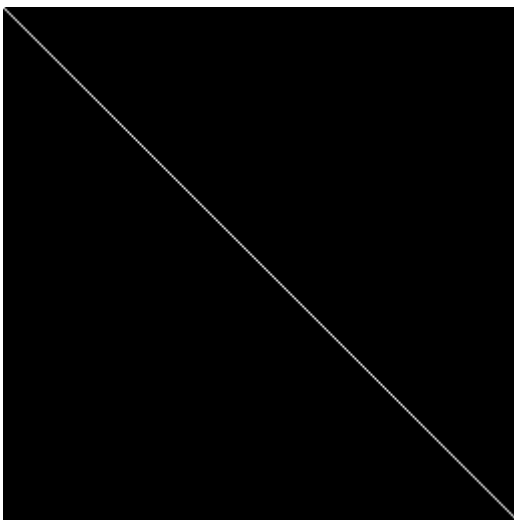
課題2では、直線を構成するすべての点を $\rho - \theta$ 空間に変換して投票を行った。各点について θ を0度から179度まで変化させながら ρ を計算し、その位置の票数を増やした。投票結果を確認すると、ある場所に票が集中していることが分かった。その中で最も票数が多い位置を探し、そのときの ρ と θ を直線のパラメータとして求めた。これにより、画像中の直線を検出することができた。

考察

課題2では、直線上のすべての点を $\rho - \theta$ 空間に投票した。その結果、票が特定の位

置に集中することが確認できた。最も票数の多い位置から ρ と θ を求めることで、画像中の直線を表すパラメータを取得することができた。課題 1 で得られた曲線が共通の位置で交わることを確認でき、Hough 変換による直線検出の仕組みを確認することができた。

課題 3



```
import cv2
```

```
import numpy as np
```

```
import math
```

```
image = cv2.imread('image09-1.png')
```

```
rho = 0
```

```
theta = 135
```

```
rad = math.radians(theta)
```

```
a = math.cos(rad)
```

```
b = math.sin(rad)
```

```
x0 = a * rho
```

```
y0 = b * rho
```

```
x1 = int(x0 + 1000 * (-b))
```

```
y1 = int(y0 + 1000 * (a))
```

```
x2 = int(x0 - 1000 * (-b))
```

```
y2 = int(y0 - 1000 * (a))
```

```
cv2.line(image, (x1, y1), (x2, y2), (255, 255, 255), 1)
```

```
cv2.imshow("result", image)
```

```
cv2.imwrite("result9-1.png", image)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

プログラムの解説

課題 3 では、課題 2 で求めた ρ と θ を使って入力画像に直線を描画した。 ρ と θ

から直線上の座標を計算し、その位置の画素を変更することで直線を表示した。最後に描画した直線と元の画像を比較し、正しく直線が検出できているかを確認した。

考察

課題3では、課題2で求めた ρ と θ を用いて直線を描画した。その結果、描画した直線は元画像の直線とほぼ重なっていた。これにより、課題2で求めた ρ と θ が正しく求められていたことを確認できた。また、Hough 変換を利用することで画像中の直線を検出できることが分かった。